

R, VS Code, Quarto, and AI Tools for DOE

AGST 50104: Experimental Design

Dr. Samuel B Fernandes

2026-01-01

Learning Objectives

By the end of this lecture, you should be able to:

- Navigate VS Code and create workspaces for reproducible research
- Understand what GitHub Copilot is and why it's useful for learning
- Use AI tools effectively for statistical programming with proper prompting
- Write effective prompts for AI assistance

Timing: ~3 minutes

Key points: - Emphasize that everyone starts somewhere—no prior R experience expected - Tools enable reproducible research and professional presentations - Growth mindset: mistakes are part of learning

Engagement: - Ask who has used R before (expect ~20-30%) - Reassure beginners that we'll build skills gradually

Why These Tools Matter

Reproducibility in Research

- **The Problem:** “It worked on my computer” is not acceptable in science
- **The Solution:** Quarto documents combine code, analysis, and narrative in one file
- **Result:** Anyone can re-run your analysis and get the same results
- **Benefit for you:** Your future self will thank you when revising a thesis chapter

Reproducible Research = Transparent Science

When you publish, reviewers and readers can verify your work. When you graduate, your advisor can extend your research.

Timing: ~4 minutes

Story to share: “I once spent 2 weeks trying to recreate an analysis from 6 months earlier. The data was there, but I couldn’t remember which transformations I applied. With Quarto, that problem disappears—the code IS the documentation.”

The Modern Research Workflow



Made with  Napkin

- R handles the analysis
- Quarto handles the communication
- AI tools help with both

Timing: ~3 minutes

Emphasize: This workflow applies whether you're writing a thesis, journal article, or grant proposal.

VS Code Setup for R & Quarto

Why VS Code for Data Science?

Key Benefits:

- Free & open-source
- Multi-language support
- **Integrated AI** (GitHub Copilot)
- Built-in Git
- Works everywhere

For This Course:

- Industry-standard workflow
- AI assistance built-in
- Easy GitHub collaboration
- Professional skill development



Tip

Alternative: RStudio also works well (see supplemental guide)

Timing: ~4 minutes

Key message: VS Code is the industry standard for modern data science. Learning it now gives you transferable skills.

Essential VS Code Extensions

Install These Three (Required):

R Extension

- **By:** REditorSupport
- R code execution
- Syntax highlighting

Quarto

- **By:** Quarto.org
- Render `.qmd` files
- Live preview

GitHub Copilot

- **AI assistance**
- Code suggestions
- Free for students

How to Install: Extensions icon (Ctrl/Cmd+Shift+X) → Search → Install

Timing: ~5 minutes

Live demo: Show how to search and install each extension. Give students 3-4 minutes to install.

VS Code Interface Tour

Five Main Areas:

1. **Activity Bar** (far left)
 - Explorer (files)
 - Search
 - Source Control (Git)
 - Extensions
 - Copilot Chat
2. **Side Bar** (left)
 - File tree
 - Search results
 - Git changes
3. **Editor** (center)
 - Your code/documents
 - Multiple tabs

- Split view
4. **Panel** (bottom)
 - **Terminal** (run R commands)
 - **Output** (Quarto render logs)
 - **Problems** (errors/warnings)
 - **Debug console**
 5. **Status Bar** (very bottom)
 - Git branch
 - File type
 - R/Python interpreter

Timing: ~4 minutes

Live demo: Open VS Code and point out each area. Show how to open terminal (Ctrl+‘ or Terminal menu).

Working with R in VS Code

Open R Terminal:

1. **Ctrl/Cmd + ‘**
2. Type **R** + **Enter**
3. Interactive R session!

Quick test:

```
2 + 2
x <- 10
x * 5
```

Run Code from Files:

- Create **.R** or **.qmd** file
- Select code
- **Ctrl/Cmd + Enter**

Tip

Pro tip: Keep R terminal open while coding

Timing: ~4 minutes

Student activity: Have everyone open an R terminal and run the test commands.

Creating a Workspace (VS Code “Projects”)

Workspaces in VS Code are like R Projects in RStudio—they keep all your files organized and settings consistent.

Create your course workspace:

1. File → Open Folder (Mac: Open...)
2. Navigate to Documents or Desktop
3. Create new folder: `doe_course`
4. Click “Select Folder” / “Open”

Recommended folder structure:

```
doe_course/  
  data/           # Your datasets  
  scripts/        # R scripts  
  analyses/       # Quarto documents  
  figures/        # Output plots  
  README.md       # Project notes
```

Timing: ~4 minutes

Student action: “Everyone create a `doe_course` workspace now. We’ll populate it together.”

GitHub Account & Copilot Setup

Why GitHub?

GitHub provides:

- **Version control** for your code (like “track changes” for scripts)
- **Collaboration** tools (share code, get feedback)
- **Portfolio** of your work (impress future employers!)
- **Free GitHub Copilot** with your .edu email
- **Backup** of all your projects in the cloud

What is GitHub Copilot?

- AI pair programmer
- Suggests code as you type
- Explains code in plain English

- Debugs errors
- Generates test data
- **Free for students!**

Timing: ~3 minutes

Emphasize: You don't need to master Git right away. Today, we're just setting up Copilot access.

Using GitHub Copilot for Statistics

What Copilot Can Do for You

Code Generation:

- Write functions from comments
- Complete code as you type
- Suggest entire code blocks
- Create plots from descriptions

Code Explanation:

- Explain what code does
- Translate between languages
- Suggest improvements
- Find bugs

Statistical Tasks:

- Run ANOVA, t-tests, regression
- Create visualizations
- Simulate data
- Apply transformations

Learning Aid:

- “How do I...?” questions
- “Why does this error occur?”
- “What's the difference between...?”

Timing: ~3 minutes

Key message: Copilot is your 24/7 coding tutor and assistant. It's incredibly powerful for learning.

Using Copilot: Two Modes

Mode 1: Inline Suggestions (as you type)

```
# Type this comment and wait:  
# Create a data frame of 4 nitrogen treatments with 6 reps each  
  
# Copilot suggests (in gray):  
treatments <- data.frame(  
  plot = 1:24,  
  nitrogen = rep(c("Control", "N50", "N100", "N150"), each = 6)  
)
```

Press **Tab** to accept | **Esc** to reject | **Alt/Option+]** for next suggestion

Mode 2: Copilot Chat (conversations)

1. Click Copilot Chat icon in Activity Bar (left)
2. Or press **Ctrl/Cmd + I** for inline chat
3. Get explanations, code, debugging help

Timing: ~4 minutes

Live demo: Show both modes. Type a comment, get suggestion. Then open chat and ask a question.

Copilot for Statistics: Essential Prompts

When writing code:

- “Generate a completely randomized design with 4 treatments and 5 replicates”
- “Create a ggplot boxplot of yield by treatment”
- “Run a one-way ANOVA on this data frame”
- “Check ANOVA assumptions with diagnostic plots”
- “Perform Tukey’s HSD post-hoc test”

When learning:

- “Explain what aov() does in R”
- “What’s the difference between Type I and II sums of squares?”
- “Why do I get this error: object not found”
- “How do I check for normality in residuals?”

Timing: ~4 minutes

Demonstrate: Show a few of these prompts in action.

Effective AI Prompting

The Anatomy of a Good Prompt

Good prompts = Clear context + Specific request + Expected format

Formula:

Context: What data/situation you have

Task: What you want to accomplish

Constraints: Any requirements or preferences

Format: How you want the output

Example:

Bad prompt: > “plot the data”

What data?

What kind of plot?

What variables?

Good prompt: > “Create a ggplot boxplot of wheat yield (kg/ha) by nitrogen treatment. The data frame is called `trial_data` with columns `treatment` and `yield`. Use a minimal theme and label axes clearly.”

Specific tool (ggplot)

Plot type (boxplot)

Variables identified

Data structure given

Style preferences

Timing: ~5 minutes

Key lesson: Specificity = better results. Give Copilot context!

Prompting Strategies for Statistics

Strategy 1: Start Simple, Then Refine

1. First prompt: “Load the tidyverse package”
2. Second prompt: “Create a data frame with treatment and yield columns”
3. Third prompt: “Calculate mean yield by treatment”
4. Fourth prompt: “Create a bar plot of mean yields”

Why this works: Each step builds on the last. Easier to debug.

Strategy 2: Provide Example Data Structure

```
# Instead of: "analyze my data"
# Do this:

# My data looks like this:
# treatment  yield  block
# Control    2400   1
# N50        2650   1
# N100       2900   1

# Run a two-way ANOVA with treatment and block as factors
```

Why this works: Copilot sees your actual variable names and structure.

Timing: ~4 minutes

Examples: Bad vs. Good Prompts

Bad Prompts:

```
# Do stats
```

- Too vague

```
# Plot
```

- No context

```
# Fix this
```

- Doesn't say what's wrong

```
# ANOVA
```

- Missing data structure

Good Prompts:

```
# Perform one-way ANOVA on yield
# by treatment using aov()
```

```
# Create ggplot histogram of  
# residuals from model_fit.  
# Add normal curve overlay.
```

```
# This code gives "object not found"  
# error. I'm trying to access the  
# yield column. Data frame is df.
```

```
# Run ANOVA on df with response =  
# yield, factor = treatment. Then  
# check assumptions with diagnostic  
# plots.
```

Timing: ~4 minutes

Activity: Have students practice writing a good prompt for a statistical task.

When to Use (and Not Use) AI

Good Uses:

- Learning syntax
- Debugging errors
- Creating visualizations
- Exploring approaches
- Getting unstuck

Don't Use For:

- Understanding **why**
- Interpreting results
- Research decisions
- Graded assessments
- Replacing learning

Caution

Rule: Always review & test AI code!

Timing: ~3 minutes

Ethical discussion: AI is a tool, not a shortcut to skip learning. Use it to learn faster, not to avoid learning.

Copilot Pro Tips for This Course

Tip 1: Give context with comments

```
# Data: 24 plots, 4 treatments, completely randomized design
# Response: wheat yield (kg/ha)
# Run ANOVA and check assumptions
```

Copilot uses ALL nearby code as context.

Tip 2: Use Copilot Chat for explanations

When you see unfamiliar code, highlight it → Right-click → “Copilot: Explain This”

Tip 3: Ask for alternatives

“Show me 3 different ways to create a randomized block design in R”

Tip 4: Request code with comments

“Generate a CRD randomization and add comments explaining each step”

Tip 5: Verify statistical appropriateness

“Is a one-way ANOVA appropriate for this experimental design? Explain why or why not.”

Timing: ~4 minutes

AI Tools Comparisons/suggestions

Core Coding & Statistics

Task	Best Tool	Why	Cost
Inline code suggestions	GitHub Copilot	Native VS Code integration; learns your style	Free (students)
Statistical explanations	ChatGPT (GPT-4)	Best at explaining concepts in plain English	Free tier or \$20/mo

Task	Best Tool	Why	Cost
R-specific help	Claude 3	Strong with data science; long context window	Free tier or \$20/mo
Complex reasoning	Claude 3 Opus	Best for experimental design decisions	\$20/mo (optional)
Quick debugging	Copilot Chat	Instant, in-editor; sees your full code	Free (students)
Learning statistics	ChatGPT/Claude	Conversational; good at analogies	Free tiers available
Code review	Claude	Detailed feedback; catches subtle issues	Free tier available

Specialized AI Tools for Agriculture & Data

Task	Best Tool	Why	Cost
Data visualization	Gamma AI	Convert raw data/tables into beautiful interactive dashboards	Free tier or \$12/mo
Visual design	Napkin	Transform text/data into diagrams, charts, infographics	Free tier available
Data analysis reports	NotebookLM	AI notebook that synthesizes & organizes information	Free (Google account)
Statistical plots	Plotly Copilot	Smart suggestions for best chart types	Free tier or paid
Latex equations	ChatGPT	Excellent at math formatting & derivations	Free tier available

Task	Best Tool	Why	Cost
Presentation design	Tome	AI-powered storytelling with slides & visuals	Free tier or \$10/mo
Data exploration	ChatGPT with Code Interpreter	Execute & visualize code output inline	\$20/mo (GPT-4)

AI Tool Strengths & Weaknesses

GitHub Copilot

- Fast inline suggestions
- Integrated in VS Code
- Learns from your code
- Free for students
- Can't explain concepts deeply
- Occasionally wrong with stats

ChatGPT (OpenAI)

- Best for learning concepts
- Great explanations
- Conversational
- Free tier (GPT-3.5)
- Sometimes verbose
- GPT-5 requires subscription

Claude (Anthropic)

- Strong with R/Python
- Very accurate with statistics
- Long conversations (memory)
- Free tier available
- Slower than Copilot
- Not integrated in editor

Gemini (Google)

- Fast responses
- Free and unlimited
- Integrated with Google products
- Less accurate for statistics
- Weaker at R coding

Timing: ~4 minutes

Recommendation: Use Copilot for coding, ChatGPT/Claude for learning concepts.

Copilot in Action: Example 1

Task: Create a completely randomized design

What you type:

```
# Generate a completely randomized design
# 4 treatments: Control, Low, Medium, High
# 6 replicates per treatment
# Assign to 24 plots randomly
```

Copilot suggests:

```
set.seed(2026)
treatments <- rep(c("Control", "Low", "Medium", "High"), each = 6)
randomized <- sample(treatments)
crd_design <- data.frame(
  plot = 1:24,
  treatment = randomized
)
```

Your job: Review, test, understand. Press Tab to accept!

Timing: ~3 minutes

Live demo: Type the comment and watch Copilot complete it in real-time.

Copilot in Action: Example 2

Task: Visualize the experimental design

What you type:

```
# Create a field layout visualization
# Show treatments in a 4x6 grid with colors
```

Copilot + Your refinement:


```
# Add coordinates
crd_design <- crd_design |>
  mutate(
    row = rep(1:4, each = 6),
    col = rep(1:6, times = 4)
  )

# Create visualization
ggplot(crd_design, aes(x = col, y = row, fill = treatment)) +
  geom_tile(color = "white", linewidth = 2) +
  geom_text(aes(label = plot), color = "white", size = 5, fontface = "bold") +
  scale_fill_manual(values = c("Control" = "#9D2235",
                              "Low" = "#FF8C42",
                              "Medium" = "#2E8B57",
                              "High" = "#4A90E2")) +
  labs(title = "Completely Randomized Design Layout") +
  theme_void(base_size = 14) +
  coord_equal()
```

Completely Randomized Design Layout

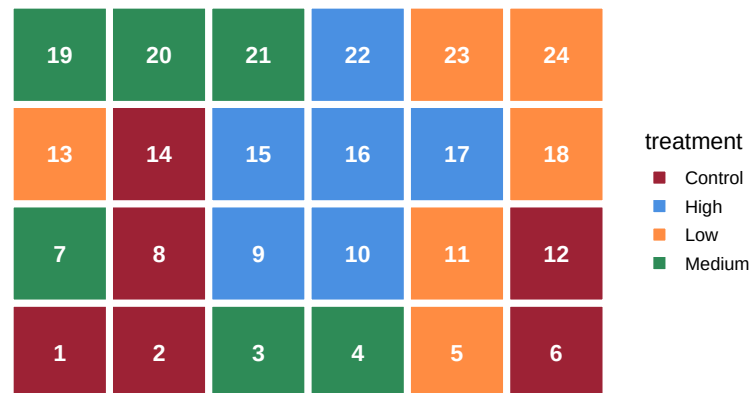


Figure 1: CRD field layout

Timing: ~4 minutes

Show: How Copilot accelerates the process, but YOU add domain knowledge (colors, labels, etc.).

ChatGPT/Claude Example: Conceptual Help

Question to ChatGPT:

“I’m designing a fertilizer experiment with 4 nitrogen rates. I have 24 plots in a field that has a known fertility gradient from north to south. Should I use a completely randomized design or a randomized complete block design? Explain the trade-offs.”

ChatGPT response (summary):

“Use a **Randomized Complete Block Design (RCBD)** for this scenario:

- **Why:** The fertility gradient is a known source of variation. Blocking by location (e.g., 6 blocks running east-west) removes this variation from experimental error.
- **Trade-off:** Slightly more complex analysis, but much better precision.
- **CRD would:** Ignore the gradient, inflating error and reducing power to detect treatment differences. Recommendation: Create 6 blocks (north to south), each containing all 4 treatments randomly assigned.”

Your takeaway: AI helps you think through experimental design decisions!

Timing: ~3 minutes

Key lesson: Use AI for “why” questions, not just “how” questions.

Summary & Key Takeaways

What We Learned Today

- **VS Code** is a powerful, free IDE for data science with R and Quarto support
- **GitHub Copilot** (free for students) provides AI-powered coding assistance
- **Workspaces** organize projects like R Projects in RStudio
- **Quarto documents** combine code, text, and output for reproducible reports
- **tidyverse** provides readable, powerful tools for data manipulation and visualization
- **AI tools** (Copilot, ChatGPT, Claude) accelerate learning and coding
- **Good prompts** = context + specific task + format
- **Randomization** in R is straightforward with `sample()` and `set.seed()`

Timing: ~3 minutes

Encouragement: “If you got Copilot working and rendered a document, you’re ahead of the curve! This setup will serve you for the entire semester.”

Resources & References

Software & Setup:

- **R:** cran.r-project.org
- **VS Code:** code.visualstudio.com
- **Quarto:** quarto.org
- **GitHub:** github.com
- **VS Code R Extension:** marketplace.visualstudio.com

Learning Materials:

- **R for Data Science (2e):** r4ds.hadley.nz
- **ggplot2 book:** ggplot2-book.org
- **Quarto Guide:** quarto.org/docs/guide
- **GitHub Copilot Docs:** docs.github.com/copilot
- **VS Code Tips:** code.visualstudio.com/docs

Resources & References

AI Tools:

- **GitHub Copilot:** github.com/features/copilot
- **ChatGPT:** chat.openai.com
- **Claude:** claude.ai
- **Gemini:** gemini.google.com

Getting Help:

- **Stack Overflow:** stackoverflow.com
- **RStudio Community:** community.rstudio.com
- **Copilot in VS Code:** Press Ctrl/Cmd+I for inline chat anytime
- **Office Hours:** See syllabus for schedule